
Introduction

Lecture 1 — Static Analysis by Abstract Interpretation

David Pichardie

Fall 2026

Roadmap

Introduction

The notion of approximation

A flavour of abstract interpretation

The Abstract Interpretation Framework

In industry: Infer

This course

§ 1

Introduction

The goal

- to prove **properties about the run-time behaviour** of a program,
- in a **fully automatic** way,
- **without actually executing** it.

A precise position

- *run-time behaviour*: unlike a **linter**, which inspects style, formatting and syntactic patterns, we reason about what the program computes when it runs.
- *automatic*: a prerequisite for scale. An analysis that needs annotations written by the programmer cannot be run on every change of a large code base.
- *without executing*: unlike **testing**, which runs the code, and only on the inputs one thought of.

What it is used for

Finding bugs

Array out of bounds, null dereference, use after free, data races, resource leaks.

Run on every code change, before review.

Security

Taint analysis: does user input reach a SQL query, a shell command, a log?

Buffer overflows: can an input-controlled size or index exceed the bounds of a buffer?

Optimisation

Constant propagation, dead code, aliasing, bounds-check elimination.

Every compiler is a static analyser.

Proof support

Invariant inference for a deductive verifier or a proof assistant — the analyser guesses, the prover checks.

Two industrial deployments

Astrée — absence of run-time errors, Airbus

Proves that the primary flight-control software of the A340/A380 raises *no* run-time error: no overflow, no division by zero, no out-of-bounds access. Zero alarms on that code base (about 1M loc of C code).

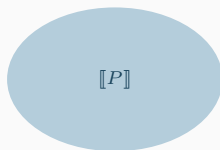
Infer — bug finding at Meta

Runs on every proposed code change, in a code base of hundreds of millions of lines, and reports to the author before review.

§2

The notion of approximation

Static analysis computes approximations



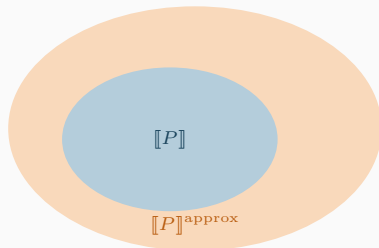
$\llbracket P \rrbracket$: the states P can really reach — **not computable**.

$\llbracket P \rrbracket^{\text{approx}}$: what the analyser computes — computable, and an over-approximation: $\llbracket P \rrbracket \subseteq \llbracket P \rrbracket^{\text{approx}}$.

ϕ_i : a set of *bad* states — computable.

- P is safe w.r.t. ϕ_1 and the analyser proves it: both intersections empty.
- P is unsafe w.r.t. ϕ_2 and the analyser says so: both intersections non-empty.
- P is safe w.r.t. ϕ_3 but the analyser **cannot prove it**: $\llbracket P \rrbracket \cap \phi_3 = \emptyset$ while $\llbracket P \rrbracket^{\text{approx}} \cap \phi_3 \neq \emptyset$.
- For ϕ_2 and ϕ_3 the analyser returns the **same** answer, “**maybe unsafe**”: from $\llbracket P \rrbracket^{\text{approx}}$ alone the two cases are indistinguishable.

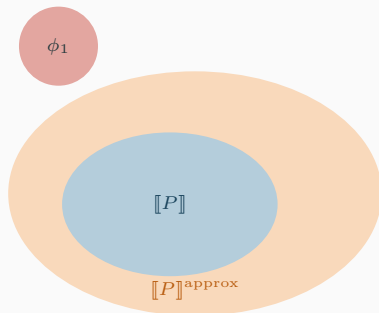
Static analysis computes approximations



$\llbracket P \rrbracket$: the states P can really reach — **not computable**.
 $\llbracket P \rrbracket^{\text{approx}}$: what the analyser computes — **computable**,
and an **over-approximation**: $\llbracket P \rrbracket \subseteq \llbracket P \rrbracket^{\text{approx}}$.
 ϕ_i : a set of *bad* states — **computable**.

- P is safe w.r.t. ϕ_1 and the analyser proves it: both intersections empty.
- P is unsafe w.r.t. ϕ_2 and the analyser says so: both intersections non-empty.
- P is safe w.r.t. ϕ_3 but the analyser **cannot prove it**: $\llbracket P \rrbracket \cap \phi_3 = \emptyset$ while $\llbracket P \rrbracket^{\text{approx}} \cap \phi_3 \neq \emptyset$.
- For ϕ_2 and ϕ_3 the analyser returns the **same** answer, **"maybe unsafe"**: from $\llbracket P \rrbracket^{\text{approx}}$ alone the two cases are indistinguishable.

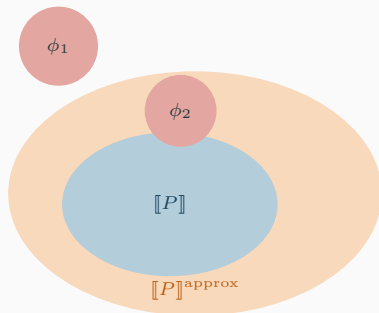
Static analysis computes approximations



$\llbracket P \rrbracket$: the states P can really reach — **not computable**.
 $\llbracket P \rrbracket^{\text{approx}}$: what the analyser computes — **computable**,
and an **over-approximation**: $\llbracket P \rrbracket \subseteq \llbracket P \rrbracket^{\text{approx}}$.
 ϕ_i : a set of *bad* states — **computable**.

- P is safe w.r.t. ϕ_1 **and the analyser proves it**: both intersections empty.
- P is unsafe w.r.t. ϕ_2 **and the analyser says so**: both intersections non-empty.
- P is safe w.r.t. ϕ_3 but the analyser **cannot prove it**: $\llbracket P \rrbracket \cap \phi_3 = \emptyset$ while $\llbracket P \rrbracket^{\text{approx}} \cap \phi_3 \neq \emptyset$.
- For ϕ_2 and ϕ_3 the analyser returns the **same** answer, "**maybe unsafe**": from $\llbracket P \rrbracket^{\text{approx}}$ alone the two cases are indistinguishable.

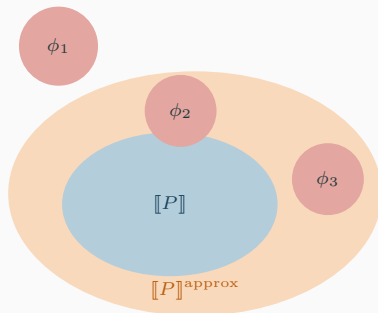
Static analysis computes approximations



$\llbracket P \rrbracket$: the states P can really reach — **not computable**.
 $\llbracket P \rrbracket^{\text{approx}}$: what the analyser computes — **computable**,
and an **over-approximation**: $\llbracket P \rrbracket \subseteq \llbracket P \rrbracket^{\text{approx}}$.
 ϕ_i : a set of *bad* states — **computable**.

- P is safe w.r.t. ϕ_1 **and the analyser proves it**: both intersections empty.
- P is unsafe w.r.t. ϕ_2 **and the analyser says so**: both intersections non-empty.
- P is safe w.r.t. ϕ_3 but the analyser **cannot prove it**: $\llbracket P \rrbracket \cap \phi_3 = \emptyset$ while $\llbracket P \rrbracket^{\text{approx}} \cap \phi_3 \neq \emptyset$.
- For ϕ_2 and ϕ_3 the analyser returns the **same** answer, “**maybe unsafe**”: from $\llbracket P \rrbracket^{\text{approx}}$ alone the two cases are indistinguishable.

Static analysis computes approximations



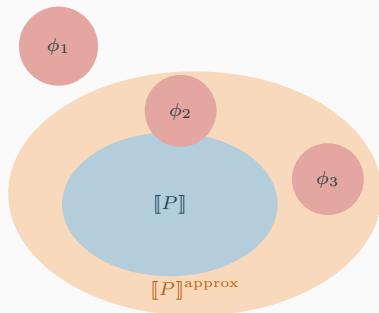
$\llbracket P \rrbracket$: the states P can really reach — **not computable**.

$\llbracket P \rrbracket^{\text{approx}}$: what the analyser computes — computable, and an **over-approximation**: $\llbracket P \rrbracket \subseteq \llbracket P \rrbracket^{\text{approx}}$.

ϕ_i : a set of *bad* states — computable.

- P is safe w.r.t. ϕ_1 **and the analyser proves it**: both intersections empty.
- P is unsafe w.r.t. ϕ_2 **and the analyser says so**: both intersections non-empty.
- P is safe w.r.t. ϕ_3 but the analyser **cannot prove it**: $\llbracket P \rrbracket \cap \phi_3 = \emptyset$ while $\llbracket P \rrbracket^{\text{approx}} \cap \phi_3 \neq \emptyset$.
- For ϕ_2 and ϕ_3 the analyser returns the **same** answer, “**maybe unsafe**”: from $\llbracket P \rrbracket^{\text{approx}}$ alone the two cases are indistinguishable.

Static analysis computes approximations



$\llbracket P \rrbracket$: the states P can really reach — **not computable**.

$\llbracket P \rrbracket^{\text{approx}}$: what the analyser computes — computable, and an **over-approximation**: $\llbracket P \rrbracket \subseteq \llbracket P \rrbracket^{\text{approx}}$.

ϕ_i : a set of *bad* states — computable.

- P is safe w.r.t. ϕ_1 **and the analyser proves it**: both intersections empty.
- P is unsafe w.r.t. ϕ_2 **and the analyser says so**: both intersections non-empty.
- P is safe w.r.t. ϕ_3 but the analyser **cannot prove it**: $\llbracket P \rrbracket \cap \phi_3 = \emptyset$ while $\llbracket P \rrbracket^{\text{approx}} \cap \phi_3 \neq \emptyset$.
- For ϕ_2 and ϕ_3 the analyser returns the **same** answer, **“maybe unsafe”**: from $\llbracket P \rrbracket^{\text{approx}}$ alone the two cases are indistinguishable.

Soundness

Every real behaviour is captured: $\llbracket P \rrbracket \subseteq \llbracket P \rrbracket^{\text{approx}}$. If the analyser reports nothing, the program is safe: no bug is missed.

Incompleteness — *false alarm*

The case of ϕ_3 : the analyser reports a danger that cannot occur. This is the price of soundness.

Note on terminology. Many industrial tools are deliberately *unsound*: they miss bugs in order to reduce noise. Both choices are defensible, but only a sound analyser proves anything.

§3

A flavour of abstract interpretation

A flavour of abstract interpretation: collecting semantics

Abstract interpretation runs the program on **sets of states** rather than states.

Collecting semantics

- A property is a subset of $\mathcal{P}(\mathbb{Z}^2)$: the possible values of (x, y) .
- Reaching a point twice, we take the **union**.
- We iterate until **stability**.

Possibly infinitely many steps — but the limit exists (Lecture 3).

```
x = 0; y = 0;
```

```
while (x<6) {
```

```
  if (?) {
```

```
    y = y+2;
```

```
  };
```

```
  x = x+1;
```

```
}
```

A flavour of abstract interpretation: collecting semantics

Abstract interpretation runs the program on **sets of states** rather than states.

Collecting semantics

- A property is a subset of $\mathcal{P}(\mathbb{Z}^2)$: the possible values of (x, y) .
- Reaching a point twice, we take the **union**.
- We iterate until **stability**.

Possibly infinitely many steps — but the limit exists (Lecture 3).

```
x = 0; y = 0;
                                {(0,0)}
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

A flavour of abstract interpretation: collecting semantics

Abstract interpretation runs the program on **sets of states** rather than states.

Collecting semantics

- A property is a subset of $\mathcal{P}(\mathbb{Z}^2)$: the possible values of (x, y) .
- Reaching a point twice, we take the **union**.
- We iterate until **stability**.

Possibly infinitely many steps — but the limit exists (Lecture 3).

```
x = 0; y = 0;           {(0,0)}
while (x<6) {
  if (?) {              {(0,0)}
    y = y+2;
  };
  x = x+1;
}
```

A flavour of abstract interpretation: collecting semantics

Abstract interpretation runs the program on **sets of states** rather than states.

Collecting semantics

- A property is a subset of $\mathcal{P}(\mathbb{Z}^2)$: the possible values of (x, y) .
- Reaching a point twice, we take the **union**.
- We iterate until **stability**.

Possibly infinitely many steps — but the limit exists (Lecture 3).

```
x = 0; y = 0;           {(0, 0)}
while (x < 6) {
  if (?) {               {(0, 0)}
    y = y + 2;           {(0, 2)}
  };
  x = x + 1;
}
```

A flavour of abstract interpretation: collecting semantics

Abstract interpretation runs the program on **sets of states** rather than states.

Collecting semantics

- A property is a subset of $\mathcal{P}(\mathbb{Z}^2)$: the possible values of (x, y) .
- Reaching a point twice, we take the **union**.
- We iterate until **stability**.

Possibly infinitely many steps — but the limit exists (Lecture 3).

```
x = 0; y = 0;           {(0, 0)}
while (x < 6) {
  if (?) {               {(0, 0)}
    y = y + 2;           {(0, 2)}
  };                     {(0, 0), (0, 2)}
  x = x + 1;
}
```

Abstract interpretation runs the program on **sets of states** rather than states.

Collecting semantics

- A property is a subset of $\mathcal{P}(\mathbb{Z}^2)$: the possible values of (x, y) .
- Reaching a point twice, we take the **union**.
- We iterate until **stability**.

Possibly infinitely many steps — but the limit exists (Lecture 3).

```
x = 0; y = 0;                               {(0, 0), (1, 0), (1, 2)}
while (x<6) {
  if (?) {
    y = y+2;                                  {(0, 0)}
                                          {(0, 2)}
  };
                                          {(0, 0), (0, 2)}
  x = x+1;                                    {(1, 0), (1, 2)}
}
```

A flavour of abstract interpretation: collecting semantics

Abstract interpretation runs the program on **sets of states** rather than states.

Collecting semantics

- A property is a subset of $\mathcal{P}(\mathbb{Z}^2)$: the possible values of (x, y) .
- Reaching a point twice, we take the **union**.
- We iterate until **stability**.

Possibly infinitely many steps — but the limit exists (Lecture 3).

```
x = 0; y = 0;                               {(0, 0), (1, 0), (1, 2)}
while (x < 6) {
  if (?) {
    y = y + 2;                               {(0, 0), (1, 0), (1, 2), ...}
  }                                           {(0, 2), (1, 2), (1, 4), ...}
                                           {(0, 0), (0, 2), ...}
  x = x + 1;                                {(1, 0), (1, 2)}
}
```

A flavour of abstract interpretation: collecting semantics

Abstract interpretation runs the program on **sets of states** rather than states.

Collecting semantics

- A property is a subset of $\mathcal{P}(\mathbb{Z}^2)$: the possible values of (x, y) .
- Reaching a point twice, we take the **union**.
- We iterate until **stability**.

Possibly infinitely many steps — but the limit exists (Lecture 3).

```
x = 0; y = 0;                               {(0, 0), (1, 0), (1, 2), ...}
while (x < 6) {
  if (?) {                                   {(0, 0), (1, 0), (1, 2), ...}
    y = y + 2;                               {(0, 2), (1, 2), (1, 4), ...}
  };                                         {(0, 0), (0, 2), ...}
  x = x + 1;                               {(1, 0), (1, 2), ...}
}
```

A flavour of abstract interpretation: collecting semantics

Abstract interpretation runs the program on **sets of states** rather than states.

Collecting semantics

- A property is a subset of $\mathcal{P}(\mathbb{Z}^2)$: the possible values of (x, y) .
- Reaching a point twice, we take the **union**.
- We iterate until **stability**.

Possibly infinitely many steps — but the limit exists (Lecture 3).

Impossible (or inefficient) to compute for all programs.

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$\{(0, 0), (1, 0), (1, 2), \dots\}$
 $\{(0, 0), (1, 0), (1, 2), \dots\}$
 $\{(0, 2), (1, 2), (1, 4), \dots\}$
 $\{(0, 0), (0, 2), \dots\}$
 $\{(1, 0), (1, 2), \dots\}$
 $\{(6, 0), (6, 2), (6, 4), \dots\}$

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ } C \text{ } 0 \wedge y \text{ } C \text{ } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;

while (x<6) {
  if (?) {

    y = y+2;

  };

  x = x+1;

}
```

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;
                                x = 0 ∧ y = 0
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$x = 0 \wedge y = 0$

$x = 0 \wedge y = 0$

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$x = 0 \wedge y = 0$

$x = 0 \wedge y = 0$

$x = 0 \wedge y > 0$ *over-approx.!*

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$x = 0 \wedge y = 0$

$x = 0 \wedge y = 0$

$x = 0 \wedge y > 0$ *over-approx.!*

$x = 0 \wedge y \geq 0$

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$x = 0 \wedge y = 0$

$x = 0 \wedge y = 0$

$x = 0 \wedge y > 0$ over-approx.!

$x = 0 \wedge y \geq 0$

$x > 0 \wedge y \geq 0$ over-approx.!

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$x = 0 \wedge y = 0 \rightarrow x \geq 0 \wedge y \geq 0$

$x = 0 \wedge y = 0$

$x = 0 \wedge y > 0$ over-approx.!

$x = 0 \wedge y \geq 0$

$x > 0 \wedge y \geq 0$ over-approx.!

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$x = 0 \wedge y = 0 \rightarrow x \geq 0 \wedge y \geq 0$

$x = 0 \wedge y = 0 \rightarrow x \geq 0 \wedge y \geq 0$

$x = 0 \wedge y > 0$ over-approx.!

$x = 0 \wedge y \geq 0$

$x > 0 \wedge y \geq 0$ over-approx.!

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$$\begin{aligned} x = 0 \wedge y = 0 &\rightarrow x \geq 0 \wedge y \geq 0 \\ x = 0 \wedge y = 0 &\rightarrow x \geq 0 \wedge y \geq 0 \\ x = 0 \wedge y > 0 &\rightarrow x \geq 0 \wedge y > 0 \\ x = 0 \wedge y \geq 0 & \\ x > 0 \wedge y \geq 0 &\text{ over-approx.!} \end{aligned}$$

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we [over-approximate](#).

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$x = 0 \wedge y = 0 \rightarrow x \geq 0 \wedge y \geq 0$

$x = 0 \wedge y = 0 \rightarrow x \geq 0 \wedge y \geq 0$

$x = 0 \wedge y > 0 \rightarrow x \geq 0 \wedge y > 0$

$x = 0 \wedge y \geq 0 \rightarrow x \geq 0 \wedge y \geq 0$

$x > 0 \wedge y \geq 0$ stable!

A flavour of abstract interpretation: approximation

Restrict the properties we manipulate, so that they stay computable.

Example: signs

$$\begin{aligned} P &::= x \text{ C } 0 \wedge y \text{ C } 0 \\ C &::= < | \leq | = | \geq | > \end{aligned}$$

Finitely many properties. To stay inside the family we **over-approximate**.

```
x = 0; y = 0;
while (x<6) {
  if (?) {
    y = y+2;
  };
  x = x+1;
}
```

$x = 0 \wedge y = 0 \rightarrow x \geq 0 \wedge y \geq 0$

$x = 0 \wedge y = 0 \rightarrow x \geq 0 \wedge y \geq 0$

$x = 0 \wedge y > 0 \rightarrow x \geq 0 \wedge y > 0$

$x = 0 \wedge y \geq 0 \rightarrow x \geq 0 \wedge y \geq 0$

$x > 0 \wedge y \geq 0$ *stable!*

$x \geq 0 \wedge y \geq 0$

Stable after two passes, and **we lost $x = 6$ on exit**: the sign domain cannot say it.

Another example: interval analysis

At each program point and for each variable, infer an interval it must belong to.

<code>assert(T.length=100); i=1;</code>	$\{i \in [1, 100]\}$
<code>while (i<T.length) {</code>	$\{i \in [1, 99]\}$
<code>p = T[i]; j = i-1;</code>	$\{i \in [1, 99], j \in [-1, 98]\}$
<code>while (0<=j and T[j]>p) {</code>	$\{i \in [1, 99], j \in [0, 98]\}$
<code>T[j]=T[j+1]; j = j-1;</code>	$\{i \in [1, 99], j \in [-1, 97]\}$
<code>};</code>	$\{i \in [1, 99], j \in [-1, 98]\}$
<code>T[j+1]=p; i = i+1;</code>	$\{i \in [2, 100], j \in [-1, 98]\}$
<code>};</code>	$\{i = 100\}$

Enough to justify every array access — as long as the bound is a constant.

Another example: polyhedral analysis

*Infer invariant linear equalities and inequalities **between** variables.*

```
assert(T.length>=1); i=1;
```

$$\{1 \leq i \leq T.length\}$$

```
while (i<T.length) {
```

$$\{1 \leq i \leq T.length - 1\}$$

```
    p = T[i]; j = i-1;
```

$$\{1 \leq i \leq T.length - 1 \wedge -1 \leq j \leq i - 1\}$$

```
    while (0<=j and T[j]>p) {
```

$$\{1 \leq i \leq T.length - 1 \wedge 0 \leq j \leq i - 1\}$$

```
        T[j]=T[j+1]; j = j-1;
```

$$\{1 \leq i \leq T.length - 1 \wedge -1 \leq j \leq i - 2\}$$

```
    };
```

$$\{1 \leq i \leq T.length - 1 \wedge -1 \leq j \leq i - 1\}$$

```
    T[j+1]=p; i = i+1;
```

$$\{2 \leq i \leq T.length + 1 \wedge -1 \leq j \leq i - 2\}$$

```
};
```

$$\{i = T.length\}$$

Now the array length is a *variable* and the proof still goes through. Precise, and expensive.

Precision and cost



Abstract interpretation as a construction kit

A domain is a lattice plus transfer functions. Once it satisfies the conditions of the framework, its soundness follows without further proof.

There is no best point on that line: the choice depends on the property to prove and on the desired precision/scalability trade-off.

§ 4

The Abstract Interpretation Framework

Patrick Cousot and Radhia Cousot

POPL 1977 — and a continuous stream ever since

A theory that unifies a large variety of static analyses

- it **formalises** the approximation of programs, so that “my analysis is correct” becomes a theorem rather than an opinion;
- it lets one **compare** the relative precision of two analyses;
- it **guides the design** of new ones — soundness by construction, rather than soundness by debugging.

Personal note

In my own research, I have applied this theory inside the Rocq proof assistant to design provably sound static analyzers.

§5

In industry: Infer

Infer, a static analyser in production

Born at Monoidics, acquired by Facebook in 2013, open source since 2015; developed at Meta ever since.

Two characteristics

- **Compositional**. It analyses a function without its callers, producing a *summary* — which is what makes it scale to hundreds of millions of lines, and what lets it re-analyse only what changed.
- Founded on **separation logic** and **bi-abduction** (Calcagno, Distefano, O'Hearn, Yang, POPL 2009): the analyser *infers* the precondition a function needs, instead of being told.

Deployment

Reports are delivered **at diff time**: on the change being proposed, to its author.

Personal note

I worked on Infer at Meta for five years.

§ 6

This course

Foundations — D. Pichardie

1. Introduction (*today*)
2. Program semantics
3. Lattices and fixpoints
4. The abstract interpretation framework
5. Abstract domains: constants and signs
6. Intervals, widening
7. Convex polyhedra
8. Octagons
9. Pointers and heap analysis

Inter-procedural — B. Montagu

10. Static call graphs (1a)
11. Static call graphs (1b)
12. Dynamic call graphs (2a)
13. Dynamic call graphs (2b)

Warning

The schedule is tight and the topic is advanced: you will have a substantial amount of work to complete in only a few weeks.

An OCaml project, in three parts, one week each

1. complete a step-by-step debugger;
2. implement backward filtering of conditions;
3. implement a small relational domain for variable equalities.

The three parts are independent. You will submit one part each week. Individual work.

<https://davidpichardie.github.io/teaching/M2/SOS/index.html>

Written exam — Fri 2 Oct

On the course. Course notes are allowed.

Oral — Fri 2 Oct

Ten minutes on your report.

Communication

Join discord.gg/NvY9QkV3H and take part in the discussions. Your engagement will be appreciated. All questions are welcome. You may use direct messages if needed.

Further reading

Optional for this course.

Getting started

- A. Miné. *Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation*. FTiPL 4(3–4), 2017.
- A. Møller, M. I. Schwartzbach. *Static Program Analysis*. Free lecture notes, cs.au.dk/~amoeller/spa/
- X. Rival, K. Yi. *Introduction to Static Analysis*. MIT Press, 2020.

The sources, and the deployments

- P. Cousot, R. Cousot. *Abstract interpretation: a unified lattice model...* POPL 1977.
- C. Calcagno, D. Distefano, P. O'Hearn, H. Yang. *Compositional Shape Analysis by means of Bi-Abduction*. POPL 2009 — the engine of Infer.
- P. O'Hearn. *Continuous Reasoning: Scaling the impact of formal methods*. LICS 2018 — why diff time.